Inference Acceleration as Closed-Loop Perturbation: Sensitivity-Guided Speedups for VLA Policies

patrick.zhang
patrick.zhang5233@gmail.com

Abstract

Inference acceleration for vision-language-action (VLA) policies is often treated as a systems problem: quantize weights, compile graphs, fuse kernels, or replay static execution paths, then report drift, success, latency, and memory. This framing is incomplete for robot policies. A VLA model is a closed-loop controller, so an implementation perturbation changes not only the current action but also the future state distribution induced by interaction with the environment. Small numerical differences can therefore be filtered, amplified, repaired, or inverted by feedback dynamics, contact, receding-horizon replanning, and thresholded task margins.

We formulate post-training VLA acceleration as a closed-loop policy perturbation problem and use this lens to derive five claims: acceleration error is filtered by closed-loop sensitivity; rollout flips are margin-crossing events; open-loop drift is insufficient without state-distribution control; sensitivity is anisotropic across action channels, rollout phases, and model boundaries; and acceleration should be viewed as exploration of a constrained policy-implementation design space rather than imitation of a single FP16 point. We test these claims primarily on a GR00T N1.5 policy in LIBERO-10 through selective W4A8 fake quantization, `torch.compile` action-head acceleration, eager-island protection, controlled action perturbations, first-divergence trace analysis, and matched rollout repair/regression accounting, then run an additional GR00T N1.7 held-out transfer check for tactic selection.

The resulting picture is not monotonic. Selective W4A8 fake quantization remains in the FP16 behavioral range over 150 matched episodes, but compensation methods repair some rollouts while regressing others. Speed-only compilation of the action head gives a 2.23× median server-latency speedup on a 33-case probe, but reduces success from 19/33 to 16/33. Sensitivity-guided protection recovers part of this loss: an early-block eager island reaches 18/33 at 2.31× speedup, and a fine duration probe finds that policy steps 0–120 restore 19/33 at 69.66 ms versus 156.22 ms for FP16. However, held-out validation shows that no single local winner is universal. Across two N1.5 30-case folds, speed-only execution is fastest but has 7 paired regressions; the 0–120 duration tactic and a combined early-block plus 0–120 tactic both reach 47/60 pooled success and 0.733 worst-fold success, with different speed–risk trade-offs. On a small N1.7 held-out check, speed-only reaches 1.48× speedup but regresses three FP16 successes, a conservative early window reaches 14/15 successes at 1.09×, and task-conditioned tactic routing reaches 1.40× while still requiring held-out validation. We do not claim final packed-kernel deployment efficiency or universal superiority of any single backend. The contribution is a modeling and experimental guide for VLA acceleration: identify closed-loop-sensitive dimensions, phases, modules, and tasks, then select tactics by paired repair/regression and latency rather than by a single probe.

1 Introduction

Large vision-language-action models are increasingly used as generalist robot policies. They map visual observations and language instructions into action chunks, often through diffusion-style denoising or transformer action heads. As these models grow, inference acceleration becomes unavoidable: deployment systems will quantize weights, compile graphs, fuse kernels, cache repeated structure, replay static CUDA graphs, or mix high- and low-precision execution. The usual evaluation asks whether the accelerated implementation is numerically close to the reference policy and faster to serve.

For closed-loop robot policies, numerical closeness is not the whole problem. An action difference at time t changes the next observation. The next action is then produced on a state that may not have been visited by the original FP16 policy. In manipulation, this feedback loop can change contact timing, grasp stability, object pose, or whether the terminal state crosses a binary success predicate. A tiny implementation difference can be absorbed in one phase, amplified near a contact boundary, or even repair a failure by nudging the rollout into a different trajectory basin.

This paper studies the following question:

> How should VLA inference acceleration be designed and evaluated when the accelerated implementation is part of a closed-loop control system?

Our answer is that post-training acceleration should be treated as a structured policy perturbation. Quantization, graph compilation, eager islands, and kernel replacement are different implementation maps from a reference policy to a constrained, faster policy. The relevant object is not only the one-step action drift on fixed observations, but also how the implementation redistributes success and failure over matched closed-loop rollouts. This changes the goal from finding a single numerically closest implementation to exploring a speed–behavior design space.

We organize the paper around five claims:

1. **Closed-loop filtering:** acceleration-induced action error matters through dynamics, policy feedback, and future closed-loop gain, not only through its norm.
2. **Margin and basin sensitivity:** rollout flips occur when perturbations cross task success margins or trajectory-basin boundaries.
3. **Open-loop insufficiency:** fixed-distribution action drift cannot predict behavior under the accelerated policy's own induced state distribution.
4. **Anisotropic sensitivity:** action channels, rollout durations, and model-layer boundaries are not equally sensitive.
5. **Solution-space exploration:** acceleration can create repairs and regressions, so the useful target is not blind FP16 imitation but sensitivity-guided search for faster implementations with better paired repair/regression trade-offs.

Our contributions are:

1. We formulate VLA inference acceleration as a closed-loop policy perturbation problem and derive local propagation, margin-crossing, and distribution-shift expressions that explain why local drift is not sufficient.
2. We turn these expressions into five experimentally testable claims and a practical guide: estimate closed-loop-sensitive dimensions, durations, and modules, then protect the smallest regions that improve paired rollout behavior while preserving speed.
3. We provide a GR00T/LIBERO behavior study showing that selective W4A8 fake quantization remains in the FP16 behavioral range while substantially redistributing which task-initialization pairs succeed.
4. We show that speed-only action-head compilation can be fast but behaviorally unsafe, and that sensitivity-guided eager islands can recover part of the loss.
5. We show that the narrow duration proxy, policy steps 0–120, is a strong but not universal local tactic: multi-fold validation and an N1.7 held-out transfer check change the conclusion from selecting one fixed winner to selecting tactics or tactic routers by worst-fold success, paired regressions, and latency.

From point solution to design-space map. The contribution is not a single deployment backend. We use quantization and compilation probes to expose the geometry of the problem: which perturbations are absorbed, which are amplified, which rollouts are repaired or regressed, and which dimensions, phases, or modules should be protected. Even readers uninterested in our exact GR00T checkpoint can use the discovered structure as a guide for designing VLA acceleration pipelines.

Model-class interpretation. Our findings are not tied to a specific checkpoint-level failure pattern. They suggest a model-class behavior: for DiT-style VLA or world-action models, small acceleration-induced implementation perturbations are filtered through closed-loop sensitivity, task margins, and trajectory basins. Therefore, larger or newer GR00T-style variants, as well as related action-diffusion policies, should be evaluated by re-estimating their sensitivity maps rather than assuming uniform acceleration tolerance across dimensions, time, or layers.

Scope of claims. This paper should be read as a behavior-level modeling and diagnostic study, not as a new quantization algorithm, compiler, or production deployment benchmark. The quantization track uses fake quantization, and the compilation track uses prototype action-head execution boundaries. We do not claim final low-precision memory, energy, or packed-kernel latency. We also do not treat small aggregate success-rate gaps as policy dominance. The main claim is methodological: VLA acceleration should be evaluated and designed as sensitivity-weighted closed-loop perturbation, using matched rollouts and paired repair/regression analysis rather than only average open-loop action error or raw serving latency.

Experimental roadmap. The experiments are intentionally staged. First, we revisit QuantVLA-style selective W4A8 fake quantization to show that compensation can reduce mean drift while still redistributing closed-loop outcomes. Second, we map sensitivity with controlled action perturbations and layer/boundary probes. Third, we use those maps to search mixed eager/compiled acceleration policies. A narrow early eager window, steps 0–120, emerges as a strong local proxy on the 33-case probe, but later held-out folds and the N1.7 transfer check show why the final object should be a robust tactic-selection protocol rather than a single fixed window.

2 Related Work

Vision-language-action policies. Recent robot policies increasingly combine large-scale visual, language, and action modeling. RT-1 introduced a scalable transformer policy trained on diverse real-robot data [2]. RT-2 framed robotic actions as tokens inside a vision-language-action model and showed transfer from web-scale vision-language training to control [3]. OpenVLA made this direction more accessible by releasing a 7B-parameter open-source VLA trained on large robot demonstration mixtures, and also noted practical interest in efficient fine-tuning and quantized serving [7]. GR00T N1 uses a dual-system VLA architecture where a vision-language module conditions a diffusion transformer action module [10]. Diffusion Policy and related action-diffusion methods model robot behavior as conditional denoising, often with receding-horizon control [4]. Our work focuses on the evaluation problem that appears when such policies are compressed or accelerated after training.

Inference acceleration for closed-loop policies. Deployment acceleration can come from low-precision arithmetic, graph compilation, operator fusion, CUDA graph replay, caching, or custom kernels. In open-loop perception or language tasks, small numerical differences are usually judged by output agreement and throughput. In VLA policies, the accelerated implementation chooses actions that alter future observations. This makes acceleration a behavioral question as well as a systems question: a faster implementation is useful only if its closed-loop policy perturbation stays within acceptable task margins.

Post-training quantization. PTQ methods for large transformers often target memory and latency while preserving model quality. LLM.int8() handles transformer outliers through mixed-precision decomposition [5]; GPTQ uses approximate second-order information for one-shot low-bit weight quantization [6]; SmoothQuant migrates quantization difficulty from activations to weights through an equivalent smoothing transform [12]; and AWQ uses activation statistics to protect salient weight channels [8]. These methods demonstrate that transformer quantization can be accurate when calibration accounts for activation structure. However, most evaluations are open-loop language, vision-language, or recognition tasks. In VLA policies, the quantized model selects actions that alter future inputs, so the calibration and evaluation target must include closed-loop effects.

Quantization for VLA models. QuantVLA is, to our knowledge, the most direct prior work on PTQ for VLA systems. It proposes a scale-calibrated PTQ framework with selective quantization, attention temperature matching, and output head balancing, and reports memory and latency gains together with LIBERO success-rate improvements [13]. Our study is not a deployment-efficiency reproduction of those claims. Instead, we use a behavior-level fake-quantization reproduction to analyze why VLA quantization can improve some rollouts while regressing others, and why paired closed-loop evaluation is necessary even when average offline drift improves.

Sequential prediction and distribution shift. The gap between offline error and closed-loop behavior is a classic issue in imitation learning and sequential prediction. DAgger formalizes the problem that future observations depend on previous actions, so policies trained or evaluated only under a fixed data distribution can suffer from compounding errors [11]. VLA acceleration creates a related but distinct problem: the policy is already trained, but a post-training implementation change perturbs its action outputs and therefore perturbs the state distribution it induces. This makes paired rollout analysis a natural complement to offline teacher-student action drift.

Feedback control perspective. Classical feedback control emphasizes that closed-loop behavior is determined by the interaction between controller perturbations, plant dynamics, and feedback gain [1]. We use this lens to interpret inference acceleration not as a static approximation problem, but as a bounded policy perturbation whose effect depends on local closed-loop sensitivity and task margins.

3 Inference Acceleration as Policy Perturbation

Let $\pi_\theta(a \mid s, l)$ denote a reference FP16 VLA policy conditioned on state observation s and language instruction l. A post-training acceleration transform constructs an implemented policy

$$\varphi = \mathcal{A}(\theta; C, S), \qquad \pi_q = \pi_\varphi, \tag{1}$$

where $\mathcal{A}$ may represent quantization, graph compilation, mixed-precision partitioning, kernel replacement, or graph replay; C denotes calibration data when applicable; and S denotes systems choices such as compilation boundaries or shape buckets. We keep the shorthand π_q for the perturbed implementation because the first experimental track is quantization, but the derivation applies to any acceleration path that induces a small action perturbation. Even without gradient updates, this is not a neutral storage transform: it changes the implemented function.

Offline action drift measures a local difference such as

$$e(s) = \|\pi_q(s, l) - \pi_\theta(s, l)\| \tag{2}$$

on observations sampled from some dataset or teacher rollout distribution. Closed-loop performance depends on the state distributions induced by the two policies, d_{π_θ} and d_{π_q}.

For a local transition model,

$$s_{t+1} = f(s_t, a_t), \qquad a_t = \pi(s_t), \tag{3}$$

a first-order error expansion gives

$$\delta s_{t+1} \approx J_s \delta s_t + J_a \delta a_t, \qquad \delta a_t = \pi_q(s_t) - \pi_\theta(s_t). \tag{4}$$

Even if δa_t is small on the teacher state distribution, the accumulated δs_t can move the accelerated policy into states where the original local error estimate no longer applies.

This is why manipulation success is not a smooth function of single-step action MSE. A rollout can flip because of a small action perturbation if that perturbation changes whether the gripper reaches the object before closing, whether a placed object crosses a success-region boundary, whether an object contact produces a different pose, or whether a receding-horizon action chunk enters a different trajectory basin. In this view, acceleration is closer to post-training policy editing than to passive systems optimization.

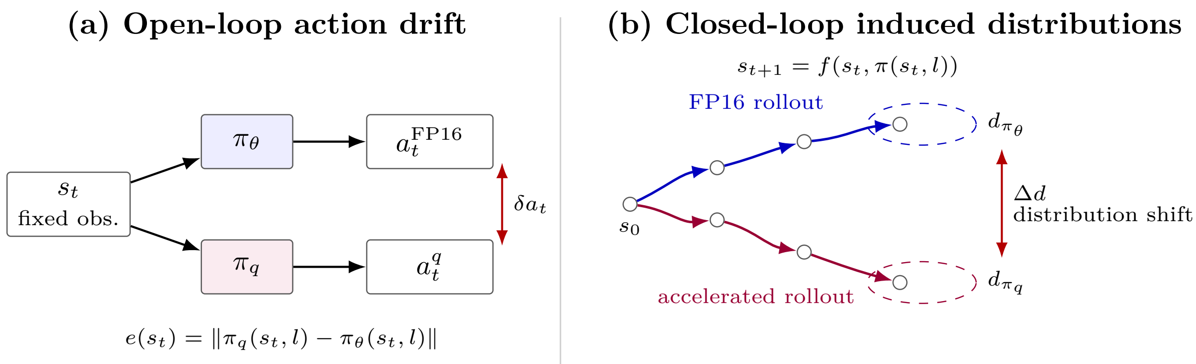

Figure 1: Inference acceleration as closed-loop policy perturbation. Offline action drift compares two actions on a fixed observation, while closed-loop execution lets each policy induce its own future observation distribution.

Claim 1: implementation error is filtered by closed-loop sensitivity. The first-order expression above can be expanded over a horizon. Let $\eta_t = \pi_q(s_t, l) - \pi_\theta(s_t, l)$ be the local acceleration-induced action perturbation on the nominal FP16 trajectory, let $A_t = \partial f/\partial s$, $B_t = \partial f/\partial a$, and $K_t = \partial \pi_\theta/\partial s$ be evaluated along that trajectory, and define the local closed-loop Jacobian

$$F_t = A_t + B_t K_t. \tag{5}$$

This claim is local and first-order: it assumes small perturbations around the FP16 trajectory, defines η_t on that nominal trajectory, and approximates the accelerated policy's local state Jacobian by the FP16 policy Jacobian, ignoring differences between $\partial \pi_q/\partial s$ and $\partial \pi_\theta/\partial s$ at this order. Under these assumptions, the terminal state perturbation satisfies

$$\delta s_T \approx \sum_{t=0}^{T-1} \left(F_{T-1} F_{T-2} \cdots F_{t+1}\right) B_t \eta_t, \tag{6}$$

with the empty product interpreted as the identity. Thus the effect of acceleration is not determined by $\|\eta_t\|$ alone, but by how its direction aligns with transition dynamics, policy feedback, and future closed-loop gain.

Claim 2: rollout flips are margin-crossing events. Let $h(s_T)$ be an implicit terminal success margin, with success when $h(s_T) > 0$. For a small terminal perturbation,

$$h(s_T^q) - h(s_T) \approx \nabla h(s_T)^\top \delta s_T. \tag{7}$$

A rollout can flip when the projected perturbation is comparable to the FP16 margin:

$$\left|\nabla h(s_T)^\top \delta s_T\right| \gtrsim |h(s_T)|. \tag{8}$$

More precisely, the outcome flips when the perturbation changes the sign of the margin:

$$\operatorname{sign}(h(s_T^q)) \neq \operatorname{sign}(h(s_T)). \tag{9}$$

If the FP16 rollout succeeds, $h(s_T) > 0$, an accelerated regression requires

$$\nabla h(s_T)^\top \delta s_T \lesssim -h(s_T). \tag{10}$$

If the FP16 rollout fails, $h(s_T) < 0$, an accelerated repair requires

$$\nabla h(s_T)^\top \delta s_T \gtrsim |h(s_T)|. \tag{11}$$

Thus Equation 8 should be read as a magnitude condition for either flip direction; the sign determines whether the perturbation causes a repair or a regression. This explains why two rollouts with similar single-step action drift can behave differently: high-margin trajectories absorb the perturbation, while low-margin trajectories near contact or placement boundaries can cross the success threshold.

Design principle: optimize sensitivity-weighted closed-loop perturbation. Equations 6 and 8 suggest that the relevant post-training acceleration objective for VLA policies is not the average open-loop action deviation

$$\min_{Q} \mathbb{E} \sum_{t} \|\eta_t(Q)\|^2, \tag{12}$$

but a sensitivity-weighted perturbation objective of the form

$$\min_{Q} \mathbb{E} \sum_{t} \eta_t(Q)^\top G_t \eta_t(Q), \tag{13}$$

where one local terminal-margin approximation is

$$G_t = B_t^\top \Phi_{t\to T}^\top \nabla h(s_T) \nabla h(s_T)^\top \Phi_{t\to T} B_t, \qquad \Phi_{t\to T} = F_{T-1} \cdots F_{t+1}. \tag{14}$$

Equation 13 is a separable local surrogate, not the exact squared full-horizon terminal-margin objective. To see this, define the scalar contribution

$$\alpha_t = \nabla h(s_T)^\top \Phi_{t\to T} B_t \eta_t = c_t^\top \eta_t, \qquad c_t = B_t^\top \Phi_{t\to T}^\top \nabla h(s_T).$$

The first-order terminal margin perturbation is $\Delta h \approx \sum_t \alpha_t$. Its exact squared approximation expands as

$$(\Delta h)^2 \approx \left(\sum_t \alpha_t\right)^2 = \sum_t \alpha_t^2 + 2 \sum_{i<j} \alpha_i \alpha_j.$$

The per-step term is $\sum_t \alpha_t^2 = \sum_t \eta_t^\top c_t c_t^\top \eta_t$, which gives Equation 13 with $G_t = c_t c_t^\top$. The omitted cross-time terms represent interactions between implementation errors at different rollout steps: they can amplify risk when their margin effects have the same sign, or partially cancel when their effects have opposite signs. We use the separable form as a tractable proxy, appropriate when cross-time error correlations are weak or when the objective is meant to rank locally sensitive perturbation directions rather than exactly optimize rollout-level margin loss. In practice G_t is difficult to estimate exactly, but the expression identifies the target: reduce implementation error in state-action directions that are amplified by feedback dynamics and projected onto small task margins. Layer-statistic corrections such as ATM/OHB and execution-boundary protection can be useful proxies, but they do not by themselves optimize this closed-loop criterion.

Claim 3: open-loop drift is incomplete without state-distribution control. Let $\ell(s)$ denote a closed-loop-relevant loss, such as action drift, sensitivity-weighted action error, or risk of approaching a failure boundary. Offline validation estimates behavior under a fixed distribution, typically d_{π_θ} or a dataset distribution. Closed-loop accelerated execution instead samples from d_{π_q}:

$$\mathbb{E}_{s\sim d_{\pi_q}}[\ell(s)] = \mathbb{E}_{s\sim d_{\pi_\theta}}[\ell(s)] + \left(\mathbb{E}_{s\sim d_{\pi_q}}[\ell(s)] - \mathbb{E}_{s\sim d_{\pi_\theta}}[\ell(s)]\right). \tag{15}$$

Open-loop action drift controls only the fixed-distribution term. It does not by itself control the distribution-shift term induced by executing the perturbed policy. This term can matter because a small nominal action error can move the system away from the FP16 trajectory, after which the accelerated policy may visit states with larger implementation error, higher closed-loop sensitivity, or smaller task margins.

More formally, if ℓ is a nonnegative bounded loss with $0 \le \ell(s) \le \|\ell\|_\infty$, then the distribution-shift contribution obeys

$$\left|\mathbb{E}_{d_{\pi_q}}[\ell] - \mathbb{E}_{d_{\pi_\theta}}[\ell]\right| \le \|\ell\|_\infty \, \mathrm{TV}(d_{\pi_q}, d_{\pi_\theta}), \tag{16}$$

under the standard definition $\mathrm{TV}(p,q) = \frac{1}{2}\|p-q\|_1$. If ℓ is L-Lipschitz under a state metric, an analogous Wasserstein bound gives

$$\left|\mathbb{E}_{d_{\pi_q}}[\ell] - \mathbb{E}_{d_{\pi_\theta}}[\ell]\right| \le L \, W_1(d_{\pi_q}, d_{\pi_\theta}). \tag{17}$$

The point is not that these distances are easy to estimate in high-dimensional robot rollouts, but that fixed-distribution drift is only one term. Predicting rollout robustness also requires controlling how far the accelerated policy moves the visited state distribution.

Claim 4: closed-loop sensitivity is anisotropic. The metric G_t implies that not all perturbations of the same norm are equally important. For a unit action coordinate e_j, an equal-magnitude perturbation $\eta_t = \epsilon e_j$ has local squared terminal-margin risk

$$r_{t,j} \approx \epsilon^2 e_j^\top G_t e_j. \tag{18}$$

Thus action channels are only equally sensitive in the special case where the relevant directional structure of G_t is uniform. In manipulation, this is unlikely: Cartesian motion, wrist orientation, and gripper timing couple differently to contact dynamics and task margins.

The same argument applies across time. For a fixed perturbation direction v injected at two rollout steps, the local risk is

$$r_t(v) \approx v^\top G_t v, \tag{19}$$

and G_t changes with B_t, $\Phi_{t\to T}$, and $\nabla h(s_T)$. Early approach, alignment, contact, grasping, transfer, and placement windows can therefore have different sensitivity even when the perturbation norm is unchanged.

Finally, for a module- or layer-level implementation perturbation $\zeta_{i,t}$ whose first-order effect on the action is $\eta_t \approx J_{i,t}\zeta_{i,t}$, the corresponding risk is

$$r_{i,t} \approx \zeta_{i,t}^\top J_{i,t}^\top G_t J_{i,t}\zeta_{i,t}. \tag{20}$$

This shows why acceleration choices cannot be based only on parameter count, FLOPs, or latency. A module, graph boundary, or duration window is attractive only if it has high speed benefit and low sensitivity-weighted risk. The practical consequence is the design rule we test later: not all action dimensions, rollout durations, or layer boundaries are equal.

Claim 5: acceleration explores a constrained solution space. If the accelerated implementation induces a different state distribution, the goal cannot be simply to mimic every FP16 rollout. FP16 is a learned policy, not an oracle. A perturbation can repair an FP16 failure by moving the system into a better trajectory basin, or regress an FP16 success by crossing a fragile margin. Aggregate success differences decompose exactly into these two directions. For matched cases $c \in \mathcal{C}$ and binary success indicators $S_A(c), S_B(c) \in \{0,1\}$,

$$\sum_{c\in\mathcal{C}} S_B(c) - \sum_{c\in\mathcal{C}} S_A(c) = \sum_{c\in\mathcal{C}} \mathbf{1}\{S_A(c)=0, S_B(c)=1\} - \sum_{c\in\mathcal{C}} \mathbf{1}\{S_A(c)=1, S_B(c)=0\}. \tag{21}$$

This identity is simple but important: a net success gain can hide substantial behavioral churn. Paired repairs and regressions are therefore not an optional diagnostic; they are the decomposition of the aggregate number.

The design implication is that acceleration is a search over constrained implementations:

$$\max_{\mathcal{A}} \;\; \mathrm{success}(\pi_{\mathcal{A}}) - \lambda\,\mathrm{latency}(\pi_{\mathcal{A}}) - \gamma\,\mathrm{risk}(\pi_{\mathcal{A}}, \pi_\theta), \tag{22}$$

subject to available quantization, compilation, precision, and kernel choices. The role of the theory is not to solve this optimization directly, but to identify useful coordinates for search: sensitive action channels, sensitive rollout windows, sensitive modules, and low-margin task slices.

4 Experimental Setup

Model and task. We use the GR00T N1.5 LIBERO long-horizon checkpoint and evaluate on LIBERO-10 [9, 10]. The accepted FP16 baseline matches the official reference result of 38/50 on the standard 5-trial LIBERO-10 run. The evaluated policy predicts action chunks with seven action components: x, y, z, roll, pitch, yaw, and gripper. Unless noted otherwise, experiments use 8 denoising steps. For the final tactic-transfer check, we also evaluate a GR00T N1.7 LIBERO-10 checkpoint on a small held-out set; this is used to test whether the tactic-search workflow, not a specific fixed tactic, carries to a newer checkpoint.

Three evidence tracks. Track A is the quantization reproduction track. It applies selective W4A8 fake quantization and optional ATM/OHB compensation to the same FP16 policy, then evaluates both open-loop action drift and closed-loop rollout redistribution. Track B is the sensitivity-identification track. It uses controlled action perturbations, first-divergence traces, and `torch.compile` action-head boundaries to locate which action channels, rollout phases, and model boundaries are closed-loop sensitive. Track C is the sensitivity-guided acceleration track. It keeps weights in FP16, compiles the action head for speed, and inserts small eager islands or duration windows as protected regions. Track C is included because deployment-oriented acceleration can introduce small numerical or graph-boundary differences even when model semantics are unchanged. We use it to test whether closed-loop sensitivity maps can guide faster implementations, not as evidence of final packed-kernel quantization speed.

Quantization scope. We study the selective `llm_dit_mlp` W4A8 fake-quantization scope: 84 selected LLM linear layers and 32 selected DiT MLP linear layers, for 116 quantized modules. DiT attention projections are intentionally left in floating point. This design matters because attention projections feed a softmax routing mechanism, while MLP errors are more often residual perturbations.

Compensation modes. Following QuantVLA terminology, we evaluate two scale-calibrated compensation mechanisms: attention temperature matching (ATM) and output head balancing (OHB) [13]. Our implementation should be read as a reproduction-oriented approximation of these mechanisms rather than a claim of exact code-level equivalence to the original implementation. ATM rescales the DiT attention query to match teacher/student attention-logit standard deviation:

$$\alpha = \frac{\mathrm{std}_{\mathrm{teacher}}(\text{attention logits})}{\mathrm{std}_{\mathrm{student}}(\text{attention logits})}, \qquad \text{query} \leftarrow \alpha \cdot \text{query}. \tag{23}$$

OHB rescales the attention output before residual addition to match teacher/student attention-output RMS:

$$\beta = \frac{\mathrm{rms}_{\mathrm{teacher}}(\text{attention output})}{\mathrm{rms}_{\mathrm{student}}(\text{attention output})}, \qquad \text{attention output} \leftarrow \beta \cdot \text{attention output}. \tag{24}$$

The combined mode applies both corrections. In all cases the compensation acts on DiT attention processors; it does not mean DiT attention weights are quantized.

Offline drift protocol. The offline validation uses 16 real LIBERO calibration observations and 128 held-out real evaluation observations. Teacher and student calls use matched random seeds because action denoising begins from random Gaussian actions. We report NMSE, relative RMSE, cosine similarity, and max absolute difference.

Closed-loop protocol. The quantization analysis evaluates LIBERO-10 initial states 0–14: 10 tasks, 15 initial states per task, and 150 episodes per policy/mode. The sensitivity-guided acceleration search starts from a 33-case matched probe spanning tasks 4, 6, and 8 with selected initial states that include easy successes, speed-only regressions, and beneficial speed-only branches. We then validate tactic selection on two additional N1.5 30-case folds over tasks 0–9, using held-out initial states 15–17 and 18–20. Finally, we run a 15-case N1.7 held-out check over tasks 0, 1, 4, 6, and 8 with initial states 24–26. For paired comparisons, every mode is evaluated on the same task-initialization pairs with deterministic policy seeds. We report Wilson 95% confidence intervals for the 150-episode aggregate table as a descriptive uncertainty estimate, but use paired repair/regression counts, first-divergence traces, EEF drift, server p50 latency, eager-request fraction, worst-fold success, and worst-fold speedup as the primary diagnostics. We use p50 serving latency as the primary speed metric because the prototype compile experiments launch isolated server processes; mean and maximum latency include cold `torch.compile`/Inductor compilation spikes and are not treated as steady-state deployment metrics.

5 Results

5.1 Offline Mean Drift Improves, But Individual Regressions Remain

On the 128 held-out real observations, ATM and OHB reduce mean action drift relative to uncompensated W4A8 (Table 1). The identity control exactly matches `none`, showing that the custom attention processor path itself does not introduce measurable drift.

Table 1: Offline teacher-student action drift on 128 held-out real LIBERO observations.

Config	Mode	Modules	NMSE mean	Rel. RMSE mean	Cosine mean
`llm_dit_mlp`	none	116	0.0178962	0.0981458	0.992492
`llm_dit_mlp`	identity	116	0.0178962	0.0981458	0.992492
`llm_dit_mlp`	ATM	116	0.0159471	0.0904183	0.993101
`llm_dit_mlp`	OHB	116	0.0160919	0.0853958	0.992769
`llm_dit_mlp`	ATM+OHB	116	0.0153168	0.0838784	0.993048

The mean improvement hides observation-level regressions. ATM+OHB has the best mean drift, while still worsening 34 of 128 held-out observations by NMSE. This is the first sign that mean offline drift is insufficient: compensation moves the action distribution closer on average, but not uniformly.

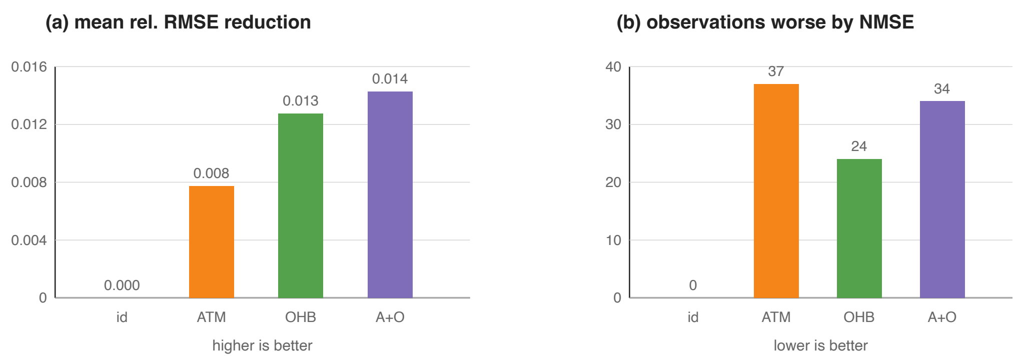

Figure 2: Offline mean drift versus observation-level regressions. Left: reduction in mean relative RMSE from uncompensated W4A8. Right: number of held-out observations whose NMSE worsens relative to uncompensated W4A8.

5.2 Selective W4A8 Remains in the FP16 Range

Over 150 LIBERO-10 task-initialization pairs, selective W4A8 remains in the same performance band as FP16 (Table 2). Some quantized variants have higher aggregate point estimates, but these gaps should not be read as evidence that quantization is inherently better than FP16. Rather, the quantized policies perturb the trajectory distribution. In finite closed-loop evaluation, that perturbation can push some failed FP16 trajectories into successful basins while pushing other successful trajectories into failure.

Table 2: Closed-loop LIBERO-10 success over task-initialization pairs 0–14.

Policy	Successes	Success rate (Wilson 95% CI)
FP16	108/150	72.0% [64.3, 78.6]
W4A8 `llm_dit_mlp` + none	113/150	75.3% [67.9, 81.5]
W4A8 `llm_dit_mlp` + ATM	114/150	76.0% [68.6, 82.1]
W4A8 `llm_dit_mlp` + OHB	116/150	77.3% [70.0, 83.3]
W4A8 `llm_dit_mlp` + ATM+OHB	114/150	76.0% [68.6, 82.1]

The confidence intervals are wide and overlapping, so the table should not be read as a statistical ranking of policies. The more stable conclusion is that selective W4A8 remains in the same behavioral range as FP16 under this protocol, while changing the identity of successful and failed rollouts.

5.3 Aggregate Rates Hide Task-Level Redistribution

The effects are task-dependent. ATM improves task 4 by 5 successes over `none`, but regresses task 8 by 5 successes. OHB is more balanced and gives the highest aggregate point estimate, but it still regresses tasks 3, 8, and 9 relative to `none`. The detailed task-level table appears in Appendix B.

5.4 Paired Rollout Flips Reveal The Mechanism

Paired comparisons over identical task-initialization pairs show that the compensation modes are not monotonic improvements (Table 3). OHB is not simply "3 episodes better" than `none`; it repairs 16 failures and introduces 13 new failures. Its net gain is small, but the underlying trajectory redistribution is substantial.

Table 3: Paired outcomes over the same 150 task-initialization pairs.

Comparison	Repaired	Regressed	Same success	Same failure	Net
ATM vs none	14	13	100	23	+1
OHB vs none	16	13	100	21	+3
OHB vs ATM	15	13	101	21	+2

Figure 3: Paired closed-loop rollout flips. Net success-rate changes decompose into repaired failures, new regressions, unchanged successes, and unchanged failures.

For the disjoint init 5–14 comparison between FP16 and ATM+OHB W4A8, we observe the same pattern: 14 FP16 failures are repaired, while 8 FP16 successes become quantized failures. The aggregate gain of 6/100 comes from this difference.

5.5 Keyframe-Based Trajectory Diagnostics

To make the paired flips more concrete, we inspected contact sheets for representative repaired and regressed rollouts. Each contact sheet contains three matched policy rows—FP16, W4A8 `none`, and W4A8 ATM+OHB—sampled uniformly over the rollout. The sheets are included in the repository keyframe directory; they are diagnostic artifacts rather than a new quantitative benchmark.

Table 4: Representative keyframe diagnostics for paired rollout flips. Frame counts refer to the number of rendered rollout frames before success or horizon failure.

Case	Outcome pattern	Keyframe-level observation	Interpretation
Task 8 init 7	FP16 fail 991; ATM+OHB fail 991; `none` success 388	`none` enters a short successful approach branch early, while FP16 and ATM+OHB spend the late rollout in repeated correction.	Raw quantization can push a weak FP16 task slice out of a failure basin.
Task 4 init 10	FP16 fail 991; `none` fail 991; ATM+OHB success 242	ATM+OHB completes the mug/plate interaction quickly; the other rows remain in horizon-length correction.	Compensation can repair a task by changing contact timing or object-interaction order.
Task 0 init 3	FP16 success 250; ATM+OHB success 267; `none` fail 991	`none` disturbs an otherwise fast object-container relation and never recovers.	The same raw perturbation that repairs some cases can regress high-margin FP16 successes.
Task 6 init 1	FP16 success 222; `none` success 477; ATM+OHB fail 991	ATM+OHB remains visually stable but under-progresses around the decisive placement phase.	Balancing can be too conservative or shift progress timing, even when it reduces mean offline drift.
Task 8 init 0	FP16 success 657; ATM+OHB fail 991; `none` fail 991	Both quantized rows take the wrong side of a task-8 decision boundary that FP16 crosses successfully.	Task-level gains are not monotonic; the same task can contain repairs and regressions.

These examples support the trajectory-redistribution interpretation. The repaired cases do not look like small final-frame placement corrections; they usually branch earlier and terminate much sooner than the corresponding failures. The regressed cases show the complementary failure: a small policy perturbation can disrupt an early object relation or produce a stable but insufficiently progressive path. Thus the paired counts in Table 3 correspond to visible differences in trajectory basins, not only bookkeeping changes in aggregate success.

5.6 Sensitivity Maps: Not All Perturbations Are Equal

The preceding results show that quantization changes which rollouts succeed. This section is the bridge from that behavioral observation to an actionable design rule. We next ask a more diagnostic question: which perturbations are closed-loop sensitive? To isolate this from implementation details of a particular quantizer, we ran small controlled perturbation probes on two FP16-success cases: task 4 init 9 and task 6 init 8. These probes should not be read as a quantization method. They are local interventions that reveal which directions, rollout windows, and module boundaries are fragile enough that a future quantization or acceleration error could matter.

Table 5: Controlled action perturbations on two FP16-success cases. All full-horizon single-channel probes use the same continuous-action perturbation norm, $\|\delta a\|_2 = 0.03$. S/F denotes success/failure and the number of policy requests; horizon failures usually reach 991 requests.

Probe	Window	Successes	Per-case outcome
No perturbation	Full	2/2	Task 4 init 9: S224; task 6 init 8: S649
6D continuous	Full	0/2	F991; F991
x only	Full	1/2	S349; F991
y only	Full	0/2	F991; F991
z only	Full	2/2	S239; S666
Roll only	Full	0/2	F991; F991
Pitch only	Full	0/2	F991; F991
Yaw only	Full	0/2	F991; F991
y only, task 4	Early / middle / late	1/3	0:75 F991; 75:150 F991; 150:225 S219
y only, task 6	Early / middle / late	2/3	0:200 F991; 200:450 S697; 450:700 S752

Table 5 supports two practical claims. First, action dimensions are not equally sensitive. A z perturbation of the same norm is absorbed in both cases, while y and rotation perturbations are catastrophic in this small probe. Second, rollout time is not equally sensitive. The same y perturbation is damaging early, but late-window perturbations can be absorbed after the task has entered a more stable phase. This maps the first two axes of the sensitivity space: channel and time. It also matches the linearized view in Section 3: the relevant quantity is not only $\|\eta_t\|$, but also the direction of η_t and the future closed-loop gain multiplying it.

We also tested whether acceleration-oriented graph boundaries are behaviorally transparent. The following experiment compiles the action-head model while leaving either no block, block 0, or block 1 as an eager island. The variants have almost identical median server latency, but their closed-loop behavior differs sharply.

Table 6: Layer/boundary sensitivity under acceleration-oriented action-head execution. The same two task-initialization pairs are used as in Table 5.

Execution mode	Successes	Task 4 init 9	Task 6 init 8	p50 latency	Speedup
FP16 baseline	2/2	S224	S649	160.53 ms	1.00×
Compile `action_head_model`	2/2	S222	S916	72.80 ms	2.21×
Compile + block 0 eager	1/2	F991	S404	72.62 ms	2.21×
Compile + block 1 eager	0/2	F991	F991	73.10 ms	2.20×

The full compiled action head is fast and succeeds on both cases, but it is not exactly behavior-transparent: task 6 takes 916 requests instead of 649. More importantly, making a small eager island does not monotonically improve safety. Block 0 eager fails task 4 but succeeds task 6 faster than the full compiled variant, while block 1 eager fails both. Because all three accelerated variants have nearly the same p50 latency, the result cannot be explained by speed alone. It is a layer/boundary sensitivity effect: changing where numerical or graph-boundary differences enter the policy can move a rollout into a different basin.

Together, the channel, time, and layer-boundary probes turn the earlier repair/regression phenomenon into a sensitivity map. These two FP16-success cases do not define a global ranking. Rather, they show that closed-loop risk is structured enough to guide what should be quantized, compiled, protected, or left in higher precision.

Finally, we compared closed-loop traces to locate the first divergence. Table 7 summarizes direct comparisons from the accelerated runs. The main pattern is temporal separation: action differences appear first, millimeter-scale state differences appear later, and outcome flips occur much later through accumulated contact and progress changes.

Table 7: First-divergence diagnostics for accelerated variants. Position thresholds are measured between matched simulator states; action thresholds use continuous-action coordinates.

Case	Pair	Outcome	First action > 0.01	First state > 1 mm	Gripper mismatch
Task 4 init 9	Full → blk0-eager	S222 → F990	Step 21	Step 59, 1.186 mm, z	Step 99
Task 4 init 9	Full → blk1-eager	S222 → F990	Step 8	Step 59, 1.012 mm, x	Step 97
Task 6 init 8	Full → blk0-eager	S916 → S404	Step 8	Step 58, 1.041 mm, x	Step 327
Task 6 init 8	Full → blk1-eager	S916 → F990	Step 53	Step 58, 1.133 mm, y	Step 261

These first-divergence traces explain why latency-equivalent implementations can have different closed-loop outcomes. The decisive failure is rarely a single large action jump. Instead, small early action differences create a delayed state separation; depending on the task margin and contact phase, that separation is either absorbed, repaired into a shorter trajectory, or amplified into horizon failure. This supports a stronger diagnostic principle: for VLA quantization and acceleration, one should map sensitivity across action channels, rollout phases, and module boundaries before choosing what to quantize, compile, or protect.

5.7 Sensitivity-Guided Acceleration Search

The previous probes identify sensitive coordinates; the next question is whether they can guide an engineering choice. We therefore expanded the proxy-guided acceleration search to a 33-case matched set. This set reuses the same task-initialization pairs for every candidate and includes speed-only regressions, baseline successes, and beneficial speed-only branches. The candidates are deliberately simple: compile the action head for speed, then protect selected layer blocks or rollout durations by falling back to eager execution.

Table 8: Layer and duration proxy search on the 33-case matched set. Server p50 is warm serving latency. Eager fraction is the fraction of server requests routed through the protected eager fallback when applicable.

Mode	Protected region	Success	Server p50	Speedup	Eager frac.
FP16 baseline	none	19/33	156.22 ms	1.00×	–
Speed-only compile	none	16/33	70.20 ms	2.23×	–
Blocks 0–3 eager	early action-head blocks	18/33	67.58 ms	2.31×	–
Duration 0–250	broad early rollout window	18/33	88.75 ms	1.76×	0.376
Duration 0–120	narrow early rollout window	19/33	69.66 ms	2.24×	0.190
Duration 0–180	early prefix	18/33	77.55 ms	2.01×	0.281
Duration 0–220	longer early prefix	16/33	90.49 ms	1.73×	0.328
Duration 80–240	contact-centered window	14/33	75.98 ms	2.06×	0.214
Duration 120–280	grasp/lift-centered window	16/33	73.48 ms	2.13×	0.216
Duration 160–240	narrow contact window	15/33	69.29 ms	2.25×	0.111
Duration 240–320	post-grasp window	17/33	66.71 ms	2.34×	0.084

Table 8 is the central probe result. Speed-only compilation gives the expected latency gain but loses three successes relative to FP16. A layer-based proxy, blocks 0–3 eager, recovers two of those successes while preserving speed. A coarse duration proxy, steps 0–250, also reaches 18/33 but costs substantially more latency. The finer duration sweep reveals the useful structure: protecting only steps 0–120 restores FP16-level success, 19/33, while preserving speed-only latency. Longer eager windows are worse. This rules out the simple explanation that more high-precision execution is safer. The sensitive window is narrow and early, but this table should be read as tactic discovery rather than as a final universal selection result.

Table 9: Paired repair/regression decomposition versus speed-only compilation on the 33-case set.

Mode	Repair	Regress	Net	Repaired cases	Regressed cases
Duration 0–250	4	2	+2	4:6, 6:0, 6:6, 8:7	6:7, 6:9
Duration 0–120	7	4	+3	4:6, 6:0, 6:3, 6:6, 8:4, 8:7, 8:10	6:7, 6:9, 6:10, 8:9
Duration 0–180	5	3	+2	4:6, 6:0, 6:6, 8:3, 8:6	6:7, 6:8, 6:9
Duration 0–220	4	4	0	4:6, 6:0, 6:6, 8:10	6:7, 6:8, 6:9, 8:9
Duration 80–240	3	5	-2	6:0, 6:6, 8:3	4:10, 6:7, 6:9, 6:10, 8:8
Duration 240–320	2	1	+1	8:3, 8:10	8:9

Table 9 explains why aggregate success alone is insufficient. The 0–120 window has the best net repair relative to speed-only, but it is not behaviorally identical to FP16. It repairs seven speed-only failures, including cases that both FP16 and speed-only fail, but it also loses four speed-only successes. Thus the accelerated implementation is not simply restored to the FP16 point; it explores a nearby closed-loop solution space.

Table 10: Selected trace diagnostics for speed-only versus the 0–120 duration proxy. The action threshold is $\ell_2 > 0.05$ in action coordinates; EEF thresholds are Euclidean position differences.

Case	Speed	0–120	First action	First EEF > 1 cm	First EEF > 5 cm	Interpretation
4:6	F990	S241	35	72	191	Returns to a short baseline-like success basin.
6:0	F990	S205	92	121	192	Returns to a baseline-like success basin.
6:6	F990	S289	86	99	136	Repairs via a longer successful branch.
8:10	F990	S765	145	154	348	Repairs a speed-only failure through a long branch.
6:3	F990	S376	54	101	299	Discovers a new success branch not found by FP16.
8:4	F990	S982	155	622	628	Near-horizon new success; visible EEF split appears late.
6:10	S225	F990	47	64	212	Main regression; early branch choice breaks a fast success.
8:9	S624	F990	52	106	170	Loses a beneficial speed-only branch.

The trace diagnostics show two mechanisms. For task 4 init 6 and task 6 init 0, the 0–120 policy stays close to a baseline-like basin; the EEF separation from baseline remains small until late in the successful rollout. For task 6 init 3 and task 8 init 4, the same early eager window produces new successful branches that neither FP16 nor speed-only found. The regressions are the complementary effect: beneficial speed-only branches are also basin choices, and a protection policy can erase them. This is why the guide must optimize paired repair/regression and latency together, not blindly minimize distance to FP16.

We then asked whether the local winner on the 33-case probe transfers to held-out initializations. Table 11 summarizes two 30-case validation folds over all LIBERO-10 tasks. Fold A uses initial states 15–17 and Fold B uses initial states 18–20. The combined tactic protects both early action-head blocks 0–3 and policy steps 0–120. Regressions are counted against the FP16 outcome on the same task-initialization pair.

Table 11: Multi-fold tactic selection on two held-out 30-case folds. A single local winner does not dominate all folds, so the selection objective must include worst-fold behavior and paired regressions in addition to speed.

Tactic	Fold A	Fold B	Pooled	Worst success	Mean speedup	Worst speedup	Regressions
Speed-only compile	25/30	20/30	45/60	0.667	2.01×	1.75×	7
Duration 0–120	22/30	25/30	47/60	0.733	1.84×	1.71×	3
Blocks 0–3 + duration 0–120	22/30	25/30	47/60	0.733	1.41×	1.07×	1

The held-out folds change the interpretation. On Fold A, speed-only compilation has the highest aggregate success, showing that an implementation perturbation can also discover beneficial branches. On Fold B, the same speed-only tactic drops to 20/30 and creates five regressions relative to FP16, while the 0–120 and combined tactics match the FP16 aggregate. The combined tactic is the most behavior-conservative candidate, with only one total paired regression across 60 cases, but it gives up speed. The 0–120 tactic is therefore preferable under a speed-constrained objective, whereas the combined tactic is preferable under a behavior-first objective. This is the practical meaning of sensitivity-guided acceleration search: the output is a Pareto choice under a validation protocol, not a proof that one protected region is universally optimal.

We repeated the final selection logic on a GR00T N1.7 checkpoint using a small held-out slice of LIBERO-10 tasks and initializations. The goal was not to prove universal transfer of a fixed window, but to check whether the same speed–risk structure appears for a newer checkpoint. Table 12 compares a speed-first tactic, conservative request-window fallbacks, and a task-conditioned tactic router. Here routing means selecting an inference tactic by task identity, not changing the robot policy: tasks 0 and 8 use `window_5_15`, while tasks 1, 4, and 6 use `speed_only`.

Table 12: GR00T N1.7 held-out tactic validation on 15 task-initialization pairs. Server p50 latency is the primary speed metric. Repairs and regressions are paired against FP16 on the same cases.

Deployment point	Success	p50 latency	Speedup	Repairs	Regressions
FP16 baseline	13/15	110.40 ms	1.00×	–	–
Speed-only compile	11/15	74.79 ms	1.48×	1	3
Request window 5–15	13/15	90.81 ms	1.22×	1	1
Request window 0–20	14/15	100.98 ms	1.09×	2	1
Request window 10–30	12/15	97.58 ms	1.13×	1	2
Task-conditioned routing	13/15	79.07 ms	1.40×	2	2

This transfer check reinforces the same lesson. If the objective is pure speed, speed-only compilation is the obvious point, but it creates the most paired regressions. If the objective is behavior-first, the conservative 0–20 window gives the highest held-out success but little speedup. Task-conditioned routing preserves most of the speed-only gain while still incurring regressions, so it should be read as a high-speed Pareto point rather than a safe default. The analogy to mixture-of-experts is useful but limited: the router selects inference tactics, not model parameters or action experts.

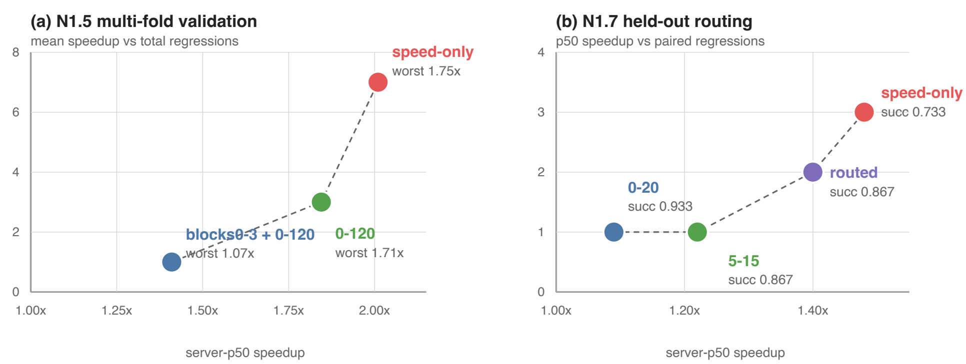

Figure 4: Speed–risk Pareto view of tactic selection. Left: N1.5 multi-fold validation shows that speed-only execution is fastest but has the largest paired regression count, while protected tactics trade speed for behavior. Right: N1.7 held-out validation shows the same structure and adds a task-conditioned tactic router as a high-speed trade-off point.

5.8 Empirical Support for the Theoretical Claims

The theoretical claims in Section 3 are not used as proof that one accelerated implementation is superior. They organize what must be measured. The experiments support the claims in the following sense.

Table 13: How the empirical results instantiate the perturbation claims.

Claim	Experimental evidence	Interpretation
Claim 1: sensitivity propagation	Small action changes produce both repairs and regressions under the same model family. Keyframe repairs branch early rather than only correcting final placement.	The effect depends on where the perturbation enters the trajectory, not only on its norm.
Claim 2: margin crossing	Task 8 contains both directions: W4A8 `none` improves task 8 from 3/15 FP16 successes to 9/15, yet task 8 init 0 is a case where both quantized modes regress a successful FP16 rollout.	The same task can contain low-margin failures that are repaired and low-margin successes that are broken.
Claim 3: open-loop insufficiency	ATM+OHB has the best mean offline drift, but OHB has the highest aggregate closed-loop point estimate; ATM improves task 4 by 5 successes over `none` while regressing task 8 by 5.	Reducing average fixed-observation error does not imply monotonic closed-loop improvement because the accelerated policy induces its own state distribution.
Claim 4: anisotropic sensitivity	Equal-norm action probes differ by channel and phase; acceleration variants with similar p50 latency differ from 2/2 success to 0/2 depending on layer boundary. In the 33-case search, duration 0–120 reaches 19/33 at 69.66 ms, while 0–220 drops to 16/33 at 90.49 ms. Across held-out folds and the N1.7 check, speed-only, duration windows, combined protection, and task-conditioned routing have different repair/regression profiles.	Perturbation risk is structured by action direction, future closed-loop gain, task margin, module location, and rollout phase. More protection is not necessarily safer.
Claim 5: solution-space exploration	Duration 0–120 restores FP16-level aggregate success on the probe, but does so by repairing 7 speed-only failures and regressing 4 speed-only successes. On held-out folds, speed-only is best on one slice and unsafe on another; on N1.7, `window_0_20` is behavior-first while task-conditioned routing is faster but not regression-free.	The goal is not to recover the FP16 point exactly or to trust one local winner. Acceleration searches a nearby implementation space where some perturbations discover better basins and others destroy beneficial branches, so tactics and tactic routers must be validated across slices.

6 Analysis

Why can acceleration improve a rollout? The accelerated policy is not guaranteed to be worse than FP16 in a finite simulator benchmark. FP16 is not an oracle; it is a learned policy with its own fragile regions. A small implementation perturbation can move a trajectory away from a failure mode. This is especially plausible when the FP16 policy is already weak on a task slice, such as LIBERO task 8 in our baseline. Therefore, an aggregate improvement should not be read as "low precision is better" or "compilation is better." It means the perturbed implementation enters different trajectory basins, some of which are more favorable under the benchmark's finite initial states and success predicates.

Why does offline drift fail to predict closed-loop behavior? Offline drift is measured on a fixed observation set. Closed-loop rollouts produce observations adaptively. Once a quantized action changes the environment state, later observations may differ from the offline distribution. This creates two mismatches: the local action error estimate may not apply to the new states, and the success function may be discontinuous around contact and placement boundaries. This explains why ATM+OHB can have the best mean offline NMSE but not the best closed-loop aggregate success.

A control-theoretic interpretation. From a feedback-control perspective, post-training acceleration acts like a structured input disturbance injected into the learned controller. For the quantization track, write

$$a_t^q = \pi_\theta(s_t^q, l) + \eta(s_t^q, l), \tag{25}$$

where η is the action perturbation induced by quantization and compensation. For a compile or kernel-replacement track, the same form applies with η denoting the action difference induced by the accelerated execution path. Linearizing around an FP16 trajectory gives

$$\delta s_{t+1} \approx (A_t + B_t K_t)\delta s_t + B_t \eta_t, \tag{26}$$

where $A_t = \partial f/\partial s$, $B_t = \partial f/\partial a$, and $K_t = \partial \pi_\theta/\partial s$ along the trajectory. The closed-loop sensitivity is therefore governed not only by the magnitude of η_t, but also by the local closed-loop gain induced by the environment dynamics and the policy feedback. A small action perturbation can be attenuated in low-gain regions and amplified near contact, grasping, or placement transitions.

This also makes success a margin problem. Let $h(s_T)$ denote an implicit terminal success margin, with success when $h(s_T) > 0$. A rollout with large positive margin can tolerate sizeable action perturbations, while a low-margin rollout can flip if acceleration moves $h(s_T)$ across zero. This explains why paired outcomes contain both repairs and regressions: an accelerated implementation can move a trajectory away from one failure basin while pushing another trajectory across a success boundary. The useful diagnostic is therefore not just average action error, but the alignment between action perturbations, closed-loop sensitivity, and task success margins.

Why is selective W4A8 robust? The `llm_dit_mlp` scope leaves DiT attention projections in floating point. This avoids quantizing the most routing-sensitive part of the action head. A perturbation to $QK^\top$ before softmax can change attention entropy and token routing. By contrast, MLP quantization more often enters as an additive residual perturbation,

$$y_{\text{student}} = y_{\text{teacher}} + \epsilon. \tag{27}$$

Layer normalization, residual connections, diffusion denoising, and receding-horizon replanning can absorb some of this error. This helps explain why uncompensated W4A8 already reaches 113/150 successes.

Why are ATM and OHB not additive? ATM changes where attention looks by changing the attention-logit scale. OHB changes how strongly the attention output enters the residual stream. If ATM moves routing into a different pattern, OHB rescales a different output than the one calibrated under the uncompensated student. The combined correction can overcorrect, undercorrect, or move the policy into a different trajectory basin. This is consistent with ATM+OHB reaching 114/150, below standalone OHB at 116/150.

What should be reported for VLA acceleration? The minimum evidence package should include offline action drift on held-out real observations, per-component action error, closed-loop success on matched task-initialization pairs, per-task and per-init success rates, paired repair/regression counts, latency and memory under the same protocol, first-divergence or EEF drift diagnostics for representative flips, and an explicit distinction between fake quantization, prototype compile paths, and final packed-kernel deployment.

7 Implications

Evaluation. VLA inference acceleration should be evaluated as a closed-loop policy perturbation. Reporting only MSE, aggregate success, or latency can be misleading. Paired rollout flips are a compact way to expose whether a faster implementation is a monotonic improvement or a redistribution of successes.

A practical guide. The experiments suggest a simple workflow for future VLA acceleration studies:

1. Build a speed-oriented candidate, such as low-precision execution, a compiled graph, or a fused kernel path, and measure both latency and matched closed-loop success.
2. Decompose the result into paired repairs and regressions instead of relying on aggregate success alone.
3. For the fragile cases, run first-divergence traces and controlled probes to identify sensitive action channels, rollout windows, and model or graph boundaries.
4. Protect the smallest candidate region—a layer island, duration window, action correction, or high-precision fallback—that improves the repair/regression balance on the probe set.
5. Re-evaluate candidate tactics on multiple held-out matched folds, because a proxy that works on one slice can overfit and destroy beneficial branches elsewhere.
6. Select a deployment tactic from the Pareto frontier using explicit constraints such as worst-fold success, maximum paired regressions, and minimum speedup.

This converts acceleration from a search for a single numerically closest implementation into an iterative exploration of the closed-loop solution space. Algorithm 1 summarizes the resulting heuristic.

Algorithm 1: Closed-Loop Sensitivity-Guided Tactic Search (CLSG-TS)

1. **Input:** candidate tactics $\mathcal{T}$, matched probe rollouts $\mathcal{P}$, measured latency $L(T_i)$, and paired repairs/regressions $R^+(T_i), R^-(T_i)$ on each validation fold.
2. Run cheap filters on each $T_i \in \mathcal{T}$: operator coverage, expected speedup, static shape compatibility, open-loop drift, layerwise drift, action-channel drift, and whether the tactic touches known contact- or basin-sensitive regions.
3. Evaluate the surviving tactics on a small matched probe set containing fragile successes, fragile failures, and speed-only branch changes. Record success, latency, first divergence, EEF drift, repairs, and regressions.
4. Estimate closed-loop risk using paired regressions, worst-case fragile failures, and sensitivity diagnostics; keep a small candidate frontier rather than a single probe winner.
5. Re-score the frontier on held-out matched folds. Select a behavior-first tactic by maximizing worst-fold success and minimizing paired regressions, or select a speed-constrained tactic by maximizing speedup subject to explicit behavior constraints. If no single tactic dominates, evaluate conditional tactic selectors over task or trajectory regimes.
6. **Output:** a behavior-first tactic, a speed-constrained Pareto tactic, or a validated tactic selector, with its matched success, regression, and latency report.

Figure 5: Heuristic search procedure used by our acceleration study. It is not an optimal-control solver; it is a practical way to spend limited rollout budget on tactics most likely to matter under closed-loop sensitivity.

Sensitivity-guided implementation search. This view is analogous to compiler tactic search, but with a different validation cost. A system such as TensorRT can enumerate many operator implementations for a fixed shape, benchmark their latency, check local numerical agreement, and choose the fastest valid tactic. For VLA acceleration, the candidate space is similar in spirit—quantization scopes, graph boundaries, eager islands, fallback windows, kernel replacements, and calibration choices are all implementation tactics—but the validation target is closed-loop behavior. Exhaustively evaluating every candidate with robot rollouts is therefore infeasible.

The practical replacement is sensitivity-guided search rather than brute-force search. Cheap static and open-loop filters should first prune candidates by latency benefit, operator coverage, layerwise drift, action-channel drift, attention-routing sensitivity, and whether the candidate touches contact-critical or basin-critical regions. A smaller set can then be evaluated on proxy rollouts containing fragile matched cases, using first-divergence, EEF drift, and paired repair/regression. Only the most promising candidates should receive larger held-out closed-loop validation. In this sense, the 0–120 duration proxy is not a universal tactic; it is a candidate discovered by sensitivity-pruned search. Table 11 shows the next step: after discovery, the tactic must be re-scored across validation folds, where behavior-first and speed-constrained objectives may choose different points on the Pareto frontier. Table 12 further shows that the same protocol can compare fixed tactics with task-conditioned tactic routing on a newer checkpoint.

Task-conditioned tactic routing. The N1.7 experiment suggests a design implication: a deployment need not use one global acceleration tactic for every task. Instead, it can select among validated tactics according to task identity or a trajectory regime. This is reminiscent of mixture-of-experts routing, but the routed objects are not model experts or learned parameters. The router selects inference tactics such as eager execution, speed-only compiled execution, or sensitivity-preserving fallback windows. In this view, acceleration becomes closed-loop-aware inference scheduling. The held-out result also shows the risk: a probe-selected router can overfit to initial states, so routing itself must be evaluated by paired held-out rollouts rather than assumed safe.

Calibration and protection. Calibration or acceleration planning should not only minimize average layerwise error or maximize local speed. It should consider which errors matter for downstream control. A useful calibration or profiling set should cover fragile contact states, low-margin success boundaries, and task slices where the FP16 policy is already weak. A control-aware objective would approximate Equation 13: weight errors by estimated closed-loop sensitivity, not only by their open-loop magnitude. This points toward rollout-informed calibration sets, residual action correction, sensitivity-guided compile boundaries, or risk-gated precision fallback as natural next steps beyond ATM/OHB-style statistic matching.

Closed-loop credit assignment. This perspective borrows an idea from reinforcement learning without requiring full policy optimization: local action errors should be assigned credit according to their downstream effect on future rollout outcomes. In standard PTQ or speed-only compilation, an error η_t is usually priced by its immediate norm or ignored if latency improves. In the closed-loop view, its risk is closer to $\eta_t^\top G_t \eta_t$, which assigns larger cost to directions that propagate through dynamics and reduce terminal task margin. This suggests a lightweight alternative to direct RL: use paired rollouts, first-divergence states, low-margin states, or repair/regression cases to estimate sensitivity proxies, then guide calibration, mixed precision, residual correction, compile boundaries, kernel replacement, or fallback decisions. The 0–120 duration proxy in Table 8 is an example of a useful proxy discovered by this process; the multi-fold comparison in Table 11 shows why it should still be selected under explicit speed–risk constraints rather than treated as universally optimal.

Sensitivity-guided acceleration. For VLA or world-model deployment, layer selection, graph compilation, and fallback windows should not be based only on parameter count, FLOPs, compiler convenience, or latency. A useful acceleration candidate should have both high compute benefit and low closed-loop sensitivity. Informally, for a module or protected region i with expected speed benefit g_i and acceleration-induced perturbation η_i, the relevant risk is closer to a sensitivity-weighted quantity,

$$r_i \approx \mathbb{E}\left[\eta_i^\top G_i \eta_i\right], \tag{28}$$

where G_i summarizes downstream dynamics, feedback, and task-margin sensitivity. This suggests a mixed-precision and mixed-execution selection rule: prioritize regions with large g_i/r_i, and leave routing-sensitive, contact-critical, basin-critical, or margin-critical regions in higher precision or eager execution unless closed-loop tests show otherwise.

The acceleration search illustrates both sides of this rule. Compiling the entire action head maximizes warm serving speed but increases closed-loop regressions. Protecting a broad early duration, 0–250, improves behavior but gives up too much latency. Protecting only 0–120 finds a better speed-constrained point in the design space, while adding early-block protection further reduces paired regressions at additional latency cost. On N1.7, a 0–20 request window is behavior-first, while task-conditioned routing is much faster but not regression-free. The deployment lesson is therefore to treat mixed precision and fallback routing as constrained search over speed benefit and closed-loop risk, not as a one-shot numerical equivalence transform.

Fine-tuning. Acceleration-aware fine-tuning is policy adaptation under deployment constraints, including low precision, compiled execution, or custom kernels. If the objective is only behavior cloning on static observations, it may still miss closed-loop state distribution shift. More policy-aware objectives may be needed, including rollout-informed calibration, DAgger-style data aggregation, or task-slice reweighting.

Deployment claims. Our results are behavior-level evidence for two prototype acceleration paths: fake W4A8 quantization and action-head compilation boundaries. They do not establish final low-precision latency, memory, or energy improvements. A complete deployment claim requires packed low-precision kernels or production compile/graph backends, plus end-to-end inference profiling under the same closed-loop protocol.

8 Limitations

This study has several limitations. The quantized implementation uses fake quantization, not a final packed int4/int8 deployment backend. The mixed-execution acceleration probes use prototype action-head compilation boundaries and duration fallbacks; they should be read as diagnostic evidence for the guide rather than a production deployment benchmark. Results are centered on one GR00T N1.5 checkpoint and LIBERO-10, with a small N1.7 held-out transfer check rather than a broad multi-family benchmark. The quantization closed-loop benchmark has 150 task-initialization pairs; the N1.5 acceleration track uses a 33-case discovery probe and two 30-case held-out folds, and the N1.7 tactic-routing check uses 15 held-out cases. These are enough to expose tactic instability but not enough for a definitive deployment benchmark. The task-conditioned router is offline-selected at task level; online sensitivity-triggered routing remains future work. We do not run real-robot experiments. ATM/OHB are implemented as reproduction-oriented calibration mechanisms; exact implementation details may differ from the original paper. The keyframe and EEF diagnostics cover representative flips, but they are not an exhaustive geometric measurement for every paired outcome.

These limitations do not weaken the central methodological point: local action approximation and closed-loop policy behavior are distinct evaluation targets.

9 Conclusion

Post-training inference acceleration of VLA policies should be viewed as closed-loop policy perturbation. Even small action errors can matter because they enter a feedback loop, alter the induced state distribution, and interact with task-dependent success margins. In our GR00T/LIBERO study, selective W4A8 fake quantization remains behaviorally robust but redistributes which rollouts succeed. Prototype action-head compilation gives substantial warm serving speedups, yet speed-only execution introduces closed-loop regressions. Sensitivity-guided search finds useful protected regions, such as the early 0–120 policy-step window, but held-out validation shows that the final choice must be made as a constrained tactic-selection problem. Behavior-first, speed-constrained, and task-routed objectives can choose different points on the same speed–risk frontier.

The main lesson is practical: VLA acceleration papers should report paired closed-loop rollouts, repair/regression decompositions, sensitivity probes, held-out tactic validation, and latency under the same protocol, not only offline drift, aggregate success, or raw serving speed. Small action errors matter when they change the trajectory basin; useful acceleration comes from finding which perturbations are closed-loop sensitive and selecting the smallest protective tactics that satisfy explicit speed–risk constraints.

A Keyframe Contact Sheets

The following contact sheets support the qualitative diagnostics in Table 4. They are enlarged here for visual inspection. Each sheet shows matched FP16, W4A8 `none`, and W4A8 ATM+OHB rows sampled uniformly over the rollout.

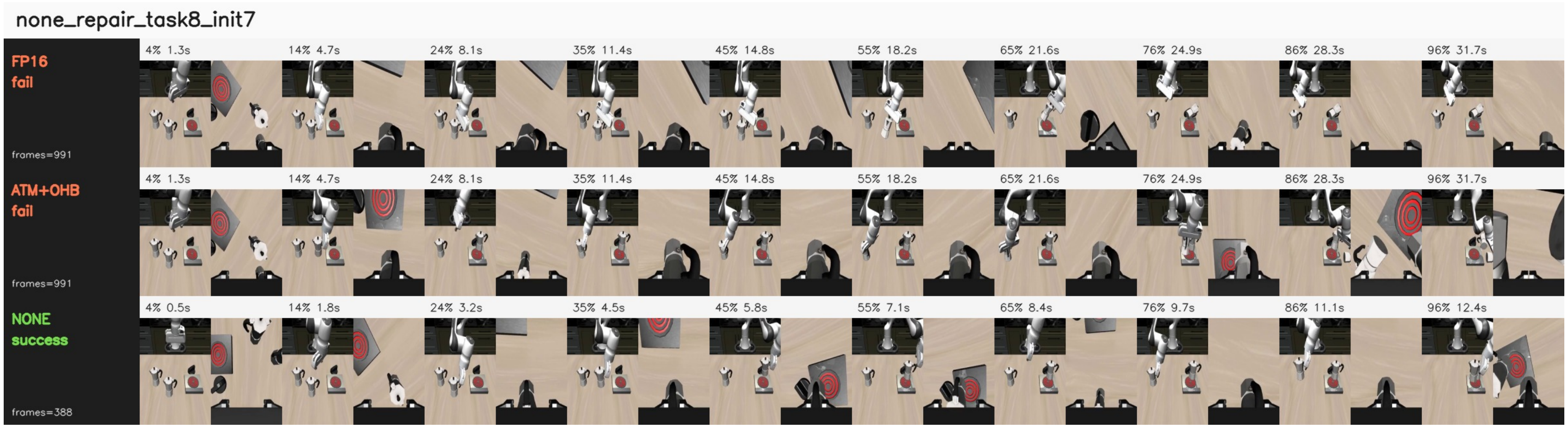

Figure 6: Task 8 init 7: W4A8 `none` repairs FP16 and ATM+OHB horizon failures by entering a shorter successful branch.

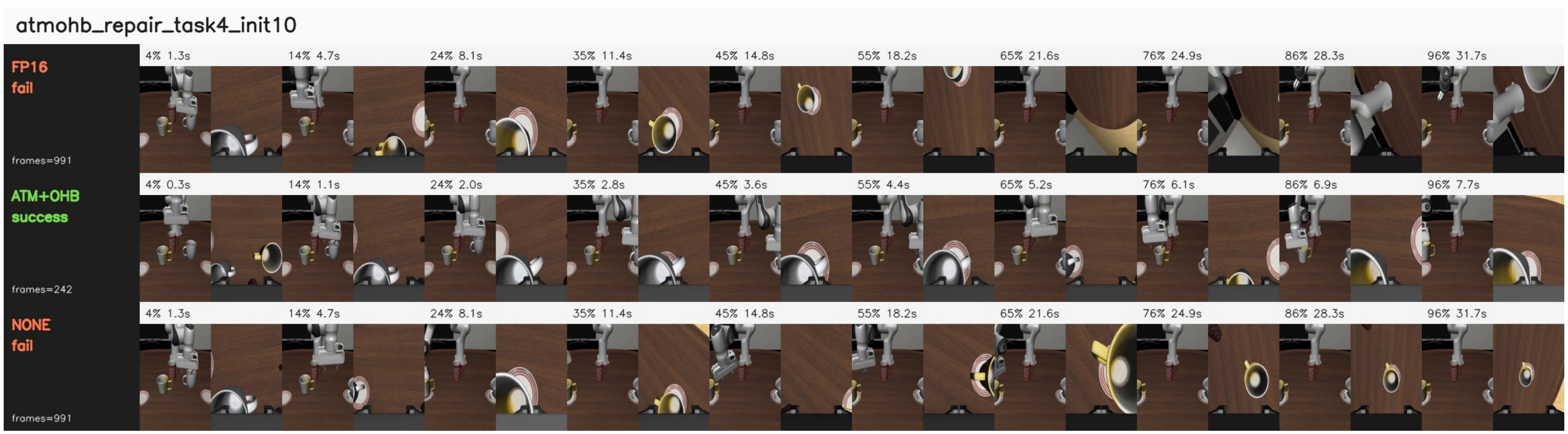

Figure 7: Task 4 init 10: ATM+OHB repairs both FP16 and W4A8 `none` failures with an early mug/plate completion.

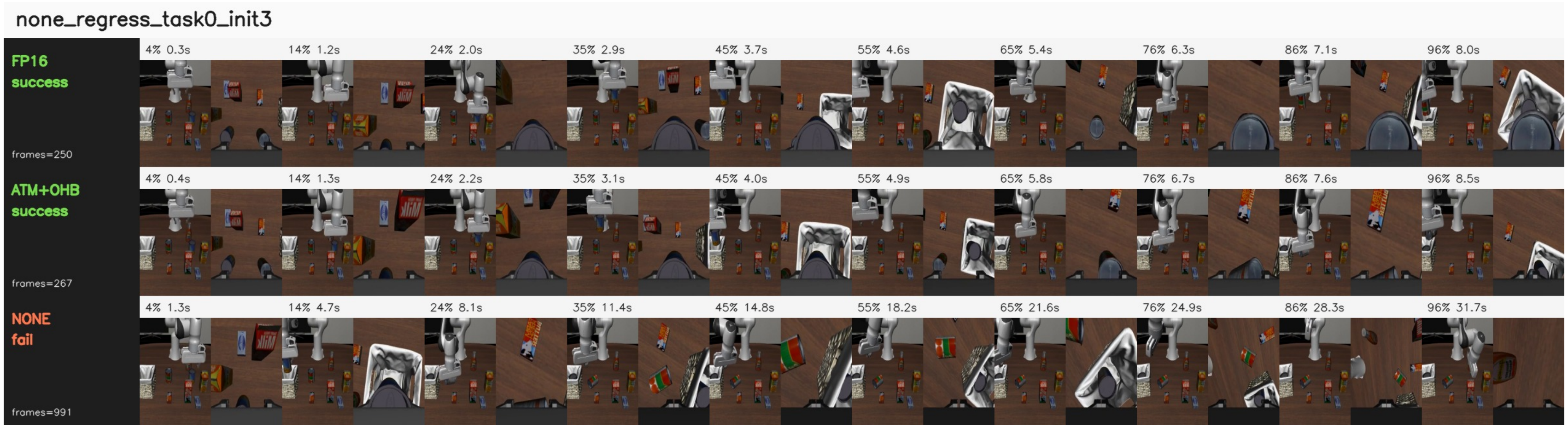

Figure 8: Task 0 init 3: W4A8 `none` regresses a fast FP16/ATM+OHB success by disturbing the early object-container relation.

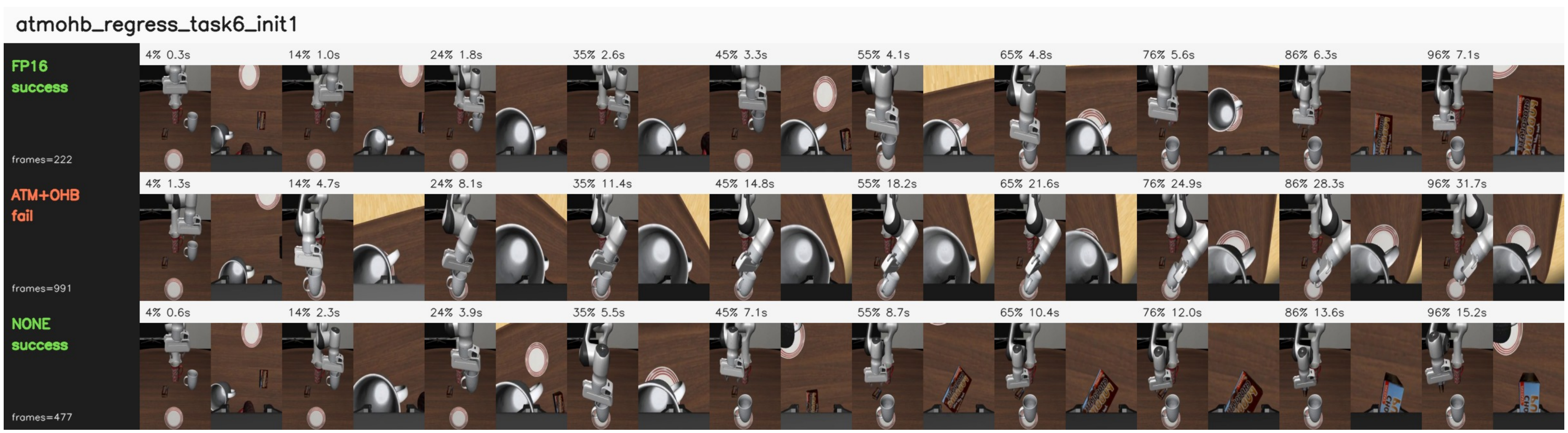

Figure 9: Task 6 init 1: ATM+OHB remains visually stable but under-progresses around the decisive placement phase, while FP16 and W4A8 `none` succeed.

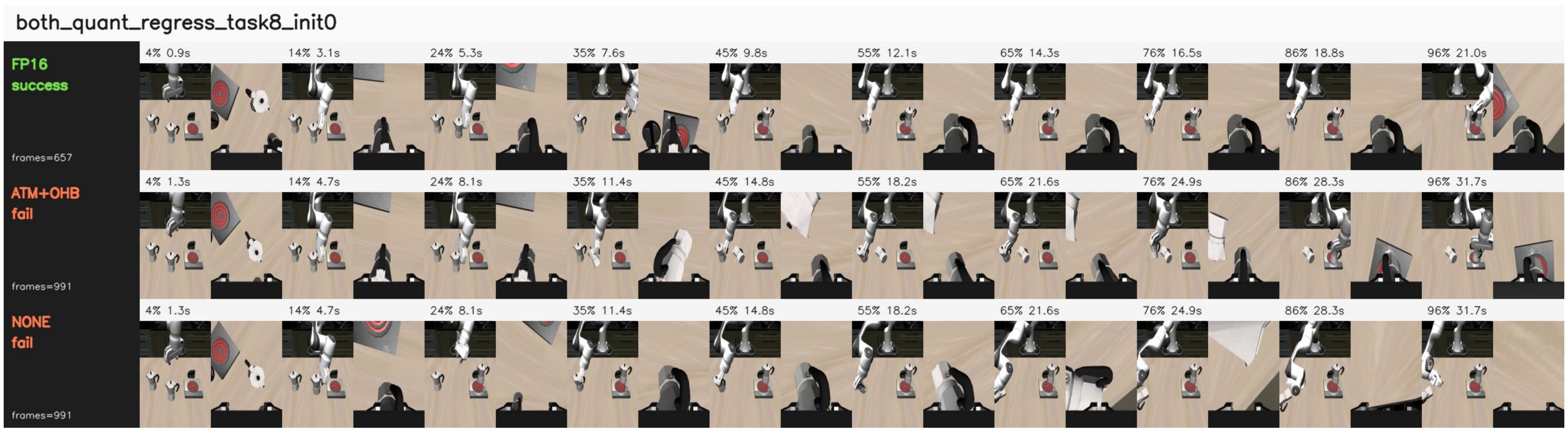

Figure 10: Task 8 init 0: both quantized modes regress a successful FP16 rollout, showing that task 8 contains both quantization repairs and quantization regressions.

B Detailed Experimental Tables

B.1 Standard 5-Trial FP16 and ATM+OHB W4A8

Table 14: Standard 5-trial LIBERO-10 comparison.

Task id	FP16	W4A8 ATM+OHB	Delta
0	5/5	5/5	0
1	3/5	4/5	+1
2	3/5	5/5	+2
3	5/5	5/5	0
4	4/5	4/5	0
5	5/5	5/5	0
6	3/5	2/5	-1
7	4/5	3/5	-1
8	1/5	0/5	-1
9	5/5	5/5	0
Total	38/50	38/50	0

B.2 Disjoint Init 5–14 FP16 and ATM+OHB W4A8

Table 15: Disjoint init 5–14 comparison.

Task id	FP16	W4A8 ATM+OHB	Delta
0	8/10	8/10	0
1	8/10	10/10	+2
2	9/10	8/10	-1
3	10/10	10/10	0
4	4/10	8/10	+4
5	10/10	10/10	0
6	3/10	5/10	+2
7	7/10	7/10	0
8	2/10	3/10	+1
9	9/10	7/10	-2
Total	70/100	76/100	+6

B.3 Ablation: W4A8 None, ATM, OHB

Table 16: Ablation over init 0–14.

Task id	None	ATM	OHB	ATM-None	OHB-None
0	10/15	11/15	13/15	+1	+3
1	13/15	13/15	14/15	0	+1
2	13/15	14/15	15/15	+1	+2
3	15/15	13/15	13/15	-2	-2
4	8/15	13/15	12/15	+5	+4
5	14/15	14/15	15/15	0	+1
6	9/15	10/15	8/15	+1	-1
7	8/15	8/15	8/15	0	0
8	9/15	4/15	6/15	-5	-3
9	14/15	14/15	12/15	0	-2
Total	113/150	114/150	116/150	+1	+3

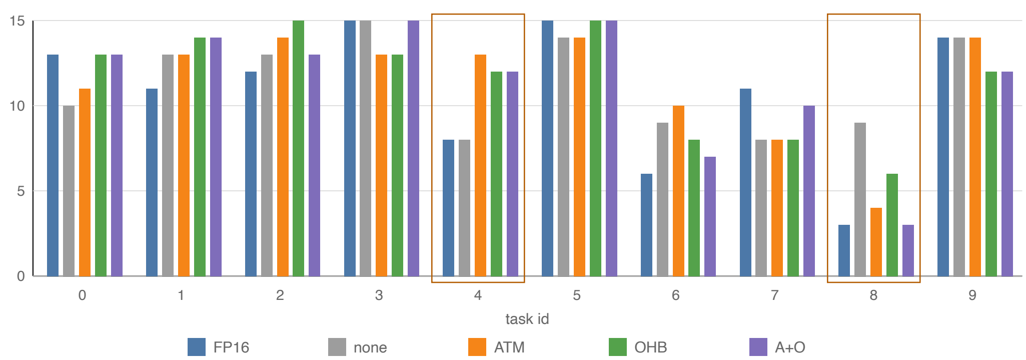

Figure 11: Task-wise success redistribution across policy variants. Aggregate changes arise from heterogeneous task-level gains and regressions rather than uniform improvement.

References

[1] Karl Johan Åström and Richard M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2008. URL `https://fbsbook.org`.

[2] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, Jasmine Hsu, et al. RT-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022. URL `https://arxiv.org/abs/2212.06817`.

[3] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023. URL `https://arxiv.org/abs/2307.15818`.

[4] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *arXiv preprint arXiv:2303.04137*, 2023. URL `https://arxiv.org/abs/2303.04137`.

[5] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. *arXiv preprint arXiv:2208.07339*, 2022. URL `https://arxiv.org/abs/2208.07339`.

[6] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022. URL `https://arxiv.org/abs/2210.17323`.

[7] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Pannag Sanketi, Quan Vuong, Thomas Kollar, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024. URL `https://arxiv.org/abs/2406.09246`.

[8] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023. URL `https://arxiv.org/abs/2306.00978`.

[9] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *arXiv preprint arXiv:2306.03310*, 2023. URL `https://arxiv.org/abs/2306.03310`.

[10] NVIDIA, Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Zhenyu Jiang, Jan Kautz, Kaushil Kundalia, Lawrence Lao, Zhiqi Li, Zongyu Lin, Kevin Lin, Guilin Liu, Edith Llontop, Loic Magne, Ajay Mandlekar, Avnish Narayan, Soroush Nasiriany, Scott Reed, You Liang Tan, Guanzhi Wang, Zu Wang, Jing Wang, Qi Wang, Jiannan Xiang, Yuqi Xie, Yinzhen Xu, Zhenjia Xu, Seonghyeon Ye, Zhiding Yu, Ao Zhang, Hao Zhang, Yizhou Zhao, Ruijie Zheng, and Yuke Zhu. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025. URL `https://arxiv.org/abs/2503.14734`.

[11] Stephane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. *arXiv preprint arXiv:1011.0686*, 2010. URL `https://arxiv.org/abs/1011.0686`.

[12] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. *arXiv preprint arXiv:2211.10438*, 2022. URL `https://arxiv.org/abs/2211.10438`.

[13] Jingxuan Zhang, Yunta Hsieh, Zhongwei Wan, Haokun Lin, Xin Wang, Ziqi Wang, Yingtie Lei, and Mi Zhang. QuantVLA: Scale-calibrated post-training quantization for vision-language-action models. *arXiv preprint arXiv:2602.20309*, 2026. doi: 10.48550/arXiv.2602.20309. URL `https://arxiv.org/abs/2602.20309`.